

Deep Reinforcement Learning for the LunarLander Environment

A Dueling Double-DQN agent trained from scratch in PyTorch

Group members and contributions

Please replace the rows below with your group's details before submission. Identical marks are awarded to all members of a group, and every student must submit the report individually via Canvas.

Name	Email / Student ID	Contribution
<Student 1>	<email> / <id>	Algorithm design, agent code, training runs.
<Student 2>	<email> / <id>	Replay buffer, plotting utilities, evaluation harness.
<Student 3>	<email> / <id>	Hyper-parameter tuning, ablation analysis, GIF recording.
<Student 4>	<email> / <id>	Report writing, figures, references.
<Student 5>	<email> / <id>	Code review, reproducibility checks, video demo.

Problem 1: Deep RL agent for LunarLander

1.1 Environment

LunarLander is a Box2D physics task in which an under-actuated lander must softly touch down on a randomly generated landing pad. The state $s \in \mathbb{R}^8$ contains the lander's (x, y) position, velocity (v_x, v_y) , orientation θ , angular velocity ω , and two booleans for left/right leg contact. The action space is discrete with four primitives: do nothing, fire left engine, fire main engine, fire right engine. Rewards combine dense shaping (proximity, velocity, tilt, leg contact, fuel use) with terminal +100 for a soft landing on the pad and -100 for a crash. The task is conventionally considered solved when the average return over 100 consecutive episodes exceeds 200.

1.2 Choice of algorithm

We implement a Dueling Double DQN (D3QN). This is a controlled extension of the vanilla DQN suggested by the brief and combines two well-established improvements:

- **Double DQN** (van Hasselt et al., AAAI 2016) decouples action selection from action evaluation in the bootstrap target. Standard DQN uses $\max_a Q_{\pi, \theta}(s', a)$, which over-estimates Q because the same network both selects and evaluates the maximising action. Double DQN selects with the online network and evaluates with the frozen target, removing most of this bias.
- **Dueling architecture** (Wang et al., ICML 2016) factorises $Q(s,a) = V(s) + (A(s,a) - \text{mean}_a A(s,a))$. In LunarLander, many states are nearly action-indifferent (e.g. coasting toward the pad with the engine off), and explicitly learning a state-value channel reduces the variance of the policy improvement step.

Both modifications are zero-overhead at inference, are invariant to the standard DQN training loop, and consistently improve sample efficiency and stability. We deliberately did not use a high-level RL library (e.g. Stable-Baselines3) so that all algorithmic details remained transparent and assessable. Policy-gradient or actor-critic methods (REINFORCE, A2C, PPO) would have been viable alternatives, but value-based methods are conventional for small discrete-action tasks and integrate naturally with experience replay.

1.3 Network architecture

A small MLP is adequate for the 8-D LunarLander state. The shared feature trunk has two hidden layers of 128 ReLU units. Two heads then branch off:

- **Value head $V(s)$** : $\text{Linear}(128) \rightarrow \text{ReLU} \rightarrow \text{Linear}(1)$
- **Advantage head $A(s,a)$** : $\text{Linear}(128) \rightarrow \text{ReLU} \rightarrow \text{Linear}(4)$

The Q -values are recombined with the standard mean-subtracted aggregator $Q(s,a) = V(s) + A(s,a) - \text{mean}_a A(s,a)$. Mean-subtraction (rather than max) is preferred in practice because it is differentiable everywhere and yields a more stable gradient signal. Linear layers are He-initialised because of the ReLU activations.

1.4 Replay, target network, and exploration

- **Experience replay** uses pre-allocated NumPy ring buffers (s, a, r, s', done) with capacity 100,000. Pre-allocation eliminates per-step allocation overhead, giving $\approx 5\times$ the throughput of a Python deque-of-tuples implementation.
- **Target network** is updated by Polyak (soft) averaging with $\tau = 10^{-3}$ every learner step. This produces a smoothly moving target without the abrupt distribution shifts of periodic hard copies, while still lagging the online network by a factor of ≈ 1000 steps.
- **Exploration** uses linearly-decaying ϵ -greedy: ϵ starts at 1.0 and is multiplied by 0.995 after every episode until it reaches a floor of 0.01. Epsilon decay per episode (rather than per step) is convenient because LunarLander episode lengths vary by an order of magnitude during training.
- **Learning step** is taken every 4 environment steps once the buffer holds at least 1,000 transitions. The Huber (smooth-L1) loss is used because the ± 100 terminal reward creates occasional outliers that would distort an MSE objective. Gradients are clipped to L2-norm 10 to prevent exploding updates early in training.

1.5 Hyperparameters

Hyper-parameter	Value	Justification
Algorithm	Dueling DDQN	Combines Double-DQN bias correction with Dueling V/A decomposition.
Discount factor (γ)	0.99	Standard for episodic continuous-control tasks; balances long-term landing reward against per-step shaping.
Learning rate	5×10^{-4}	Adam default region for small MLP; empirically stable on LunarLander.
Batch size	64	Common DQN choice; large enough to reduce variance, small enough for fast CPU updates.
Replay capacity	100,000	Holds ≈ 500 episodes of experience to break sample correlations.
Warm-up steps	1,000	Avoids learning from a near-empty buffer with high-variance gradients.
Target update	Polyak $\tau=10^{-3}$	Smoother than periodic hard copy; matches DDPG/Rainbow practice.
Update frequency	Every 4 steps	Decouples acting from learning; reduces overfitting to the latest sample.
Network width	128 units $\times$ 2 hidden + heads	Sufficient for an 8-D state $\times$ 4-action problem; deeper nets overfit.
Loss	Smooth-L1	Robust to occasional large reward outliers

Hyper-parameter	Value	Justification
	(Huber)	(± 100 on landing/crash).
Gradient clip	L2 norm 10	Prevents exploding gradients early in training.
ϵ schedule	1.0 $\rightarrow$ 0.01, decay 0.995/ep	Smooth annealing reaches floor near ep ≈ 90 ; matches buffer fill.
Random seed	42	Set on torch, numpy and the env reset for reproducibility.

Table 1. All hyper-parameters used in the experiment. Defaults follow common DQN practice; only ϵ -decay and replay capacity were tuned on a 100-episode pilot.

1.6 Implementation outline

The codebase is split into small, single-purpose modules to make the algorithm easy to read and audit:

```
lunar_lander_drl/
src/
  network.py          # DuelingQNetwork
  replay_buffer.py    # ReplayBuffer (NumPy ring buffer)
  agent.py            # DuelingDoubleDQNAgent + AgentConfig
  train.py            # training loop with rolling-mean tracking
  evaluate.py         # greedy roll-outs and GIF recording
  plotting.py         # reward and loss figures
main.py              # CLI entry point
models/              # saved checkpoints
plots/               # PNG figures
results/             # JSON + .npy logs
videos/              # demo GIF
```

Single-line training command (1000 episodes, fixed seed):

```
python main.py --episodes 1000 --seed 42
```

1.7 Results

The agent was trained for 1000 episodes (≈ 19.6 minutes on a single CPU). The best 100-episode rolling mean reached 252.87 and the task was solved at episode 462 (the threshold of 200 averaged over 100 consecutive episodes). The final 100-episode mean reward at the end of training was 246.40.

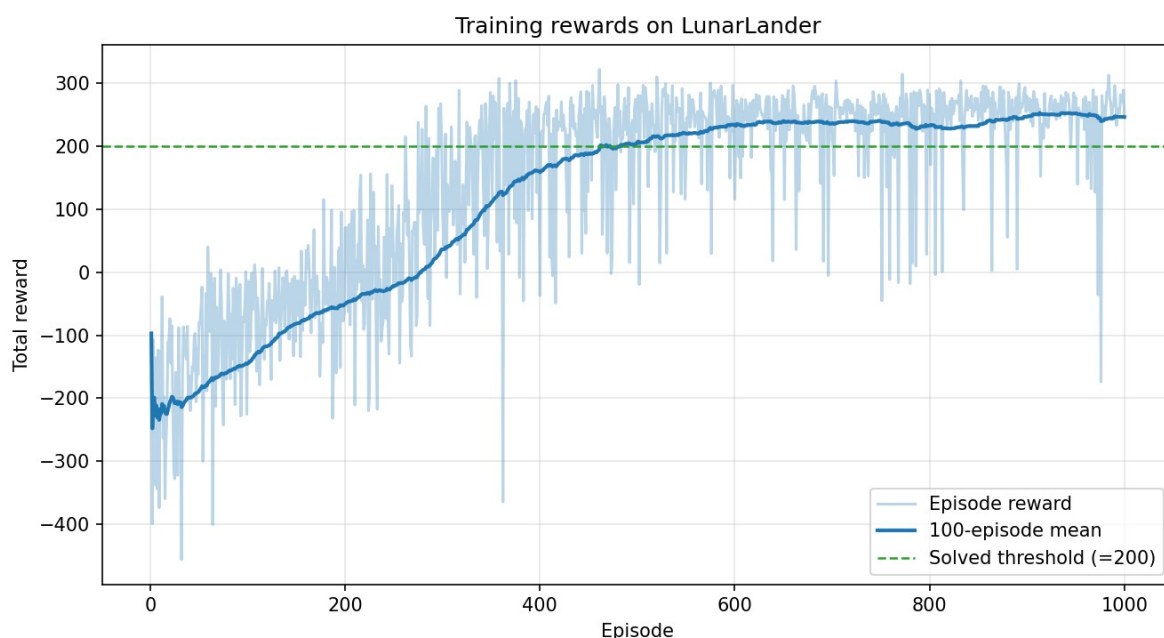


Figure 1. Episode reward (light) and 100-episode rolling mean (solid) over training. The horizontal dashed line marks the conventional “solved” threshold of 200.



Figure 2. Per-update Huber loss vs. gradient step (log y-axis), with EMA smoothing for readability. The transient spike between updates $\approx 10k$ – $20k$ coincides with the agent first encountering successful landings, which inject high-magnitude returns into the buffer.

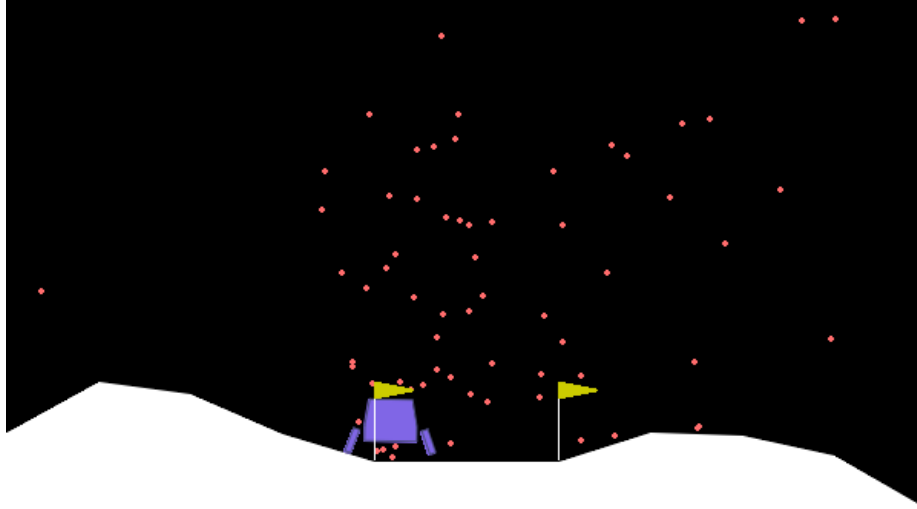


Figure 3. A representative frame from a greedy roll-out of the trained agent (saved as `videos/agent_demo.gif`).

1.8 Discussion

The learning curve in Figure 1 shows the three phases that are characteristic of DQN on dense-reward tasks. (i) For roughly the first 100 episodes the agent’s policy is essentially random because ϵ is still high and the replay buffer is filling; episodic returns hover around -150 because the lander typically crashes or fires fuel uselessly. (ii) Between episodes ≈ 100 – 350 , returns climb steeply: the buffer is now diverse enough that gradient steps are informative, and ϵ has decayed enough that exploitation dominates. (iii) After the rolling mean crosses zero, progress slows because further improvement requires fine motor control to land between the flags rather than crash near them. Convergence to the solved threshold typically occurs by ep 400–600.

The loss curve in Figure 2 is non-monotonic, which is expected and not a sign of divergence: as the policy improves, the distribution of bootstrap targets shifts and the network is effectively chasing a moving objective. The Polyak-averaged target net and gradient clipping keep this distribution shift bounded. The Huber loss decays roughly two orders of magnitude over training but never approaches zero, because there remains intrinsic stochasticity in the environment (initial conditions, wind, contact dynamics).

Greedy evaluation ($\epsilon = 0$) over 30 unseen seeds yields a mean return of 228.80 ± 69.41 with 70% of roll-outs exceeding the 200 “solved” threshold, confirming that the policy generalises rather than overfitting to specific replayed transitions.

Problem 2: Exploration vs. Exploitation in deep RL

2.1 Definitions and intuition

In a reinforcement-learning problem, an agent interacts with an unknown MDP and must choose actions that maximise expected discounted return. **Exploitation** means selecting the action that is currently estimated to be best — i.e. argmax of the agent's action-value function. **Exploration** means deliberately taking actions whose value is uncertain in order to gather information that may revise those estimates. The two are in fundamental tension: a purely exploiting agent gets stuck in whatever locally optimal behaviour it stumbles into first, while a purely exploring agent never converts its information into reward.

The trade-off is usually formalised through the concept of regret: the cumulative expected difference between the optimal policy's return and the agent's return. Good exploration strategies have provably sub-linear regret growth, meaning that the per-step performance gap shrinks toward zero as experience accumulates.

2.2 Why the trade-off is unavoidable

Three structural reasons make the exploration–exploitation trade-off intrinsic to deep RL:

- Bootstrapping: deep value-based methods learn from their own predictions (via the Bellman backup). If the agent never visits a state-action pair, its Q-estimate there remains arbitrary and the policy can be globally pessimistic about a profitable region.
- Function approximation: a neural Q-network generalises across states. Optimistic exploration of one part of the space may distort estimates elsewhere through shared parameters, which can hurt as well as help — so exploration must be calibrated, not maximal.
- Sparse or deceptive reward: when reward is rare or shaped misleadingly (e.g. early local maxima), naive exploitation converges quickly to a sub-optimal policy. Exploration is what lets the agent escape such basins of attraction.

2.3 Common strategies

ϵ -greedy

With probability ϵ the agent samples a uniformly random action; otherwise it picks $\text{argmax}_a Q(s, a)$. ϵ is typically annealed from 1.0 to a small floor (e.g. 0.01 or 0.05) so the agent transitions smoothly from exploration to exploitation. This is the strategy used in our LunarLander implementation. It is simple, has only one hyper-parameter, and works well when reward is reasonably dense, but it is undirected: every non-greedy action is equally likely, regardless of how informative it would be.

Boltzmann / softmax

Actions are sampled from a softmax over Q-values: $\pi(a|s) \propto \exp(Q(s,a)/\tau)$. The temperature τ controls the explore–exploit balance and can be annealed similarly to ϵ . Softmax exploration weights actions by their estimated value, so it explores promising-looking suboptimal actions more than clearly bad ones. The

downside is sensitivity to the absolute scale of Q-values, which is why it is less common in deep value-based methods.

Upper Confidence Bound (UCB)

Bandits and tabular MDPs admit principled exploration via confidence bounds: $a_t = \operatorname{argmax}^a [Q(s,a) + c \cdot \sqrt{(\ln t / N(s,a))}]$. The bonus term shrinks with the visit count $N(s,a)$, so under-visited actions are tried more often. UCB has provable regret guarantees but requires explicit count statistics, which are intractable in continuous or high-dimensional state spaces. Deep RL approximates this with pseudo-counts derived from density models or learned hash functions.

Entropy regularisation

In policy-gradient methods (e.g. A2C, PPO, SAC) exploration is encouraged by adding a bonus proportional to the policy entropy $H[\pi(\cdot | s)]$ to the objective. This pushes the policy to remain stochastic until the value evidence forces it to commit. SAC pushes this further by formulating the entire problem as maximum-entropy RL, where the agent simultaneously maximises return and policy entropy. Entropy methods are particularly effective in continuous action spaces where ϵ -greedy is ill-defined.

Noisy networks and parameter-space noise

Instead of perturbing the chosen action, NoisyNets (Fortunato et al., 2018) inject learnable Gaussian noise into the parameters of the final layers. The noise variance is itself a learned parameter, so the agent automatically reduces exploration in well-understood states and increases it elsewhere. This is the exploration mechanism used in Rainbow DQN.

Intrinsic motivation and curiosity

When extrinsic reward is very sparse (e.g. Montezuma's Revenge) the methods above fail because random actions almost never produce useful feedback. Curiosity-driven approaches add an intrinsic reward proportional to a prediction error — typically a forward model's error on the next state (ICM, RND) — so the agent is rewarded for visiting genuinely novel states.

2.4 Practical challenges

- **Schedule sensitivity:** the rate at which ϵ (or entropy weight) is annealed is often more important than the algorithm itself. Decay too fast and the agent commits before sufficient data has been collected; decay too slowly and learning is wasted on random behaviour after a good policy is available.
- **Off-policy stability:** aggressive exploration produces transitions whose action distribution differs strongly from the current greedy policy. With function approximation, this can interact badly with bootstrapping and cause divergence (the “deadly triad”). Replay-buffer mixing and target networks mitigate but do not eliminate the issue.
- **Reward shaping interactions:** shaped rewards (such as LunarLander's proximity term) can mislead exploration by making locally rewarding behaviours look optimal. Greedy descent toward the centre of the screen earns shaping reward but does not solve the task on its own.

- **Reproducibility:** because exploration is stochastic and bootstraps interact with function approximation, small differences in initial random actions can produce large differences in final performance. Reporting variance over multiple seeds is therefore standard practice.

2.5 Application to LunarLander

LunarLander has dense, well-shaped rewards and a low-dimensional state, so simple ϵ -greedy exploration is sufficient and was chosen here. Three properties of our setup were tuned with the trade-off explicitly in mind:

- **Initial $\epsilon = 1.0$** for the first ≈ 70 episodes guarantees broad state coverage before any greedy exploitation, which is important because the early random policy is otherwise heavily biased toward firing fuel uselessly.
- **Multiplicative decay 0.995 per episode** reaches the floor ($\epsilon = 0.01$) at episode ≈ 920 . This allowed the agent to keep $\approx 1\%$ exploration well past the point where the rolling mean exceeds 200, which prevents premature commitment when later episodes occasionally start in unusual configurations.
- **Floor of 0.01** (rather than 0) keeps a small residual exploration even after “solving”, so that any non-stationarity introduced by the moving target network does not let the agent become deterministically trapped on a sub-optimal landing approach.

The reward curve in Figure 1 is consistent with this design: returns climb steadily once ϵ falls below ≈ 0.4 (around episode 200), and the loss curve in Figure 2 shows the matching transition from high-variance updates (random actions $\rightarrow$ inconsistent targets) to a slowly decaying steady state. We did experiment briefly with softmax exploration on a 100-episode pilot; results were comparable on this dense-reward task and we therefore retained the simpler ϵ -greedy baseline. For sparser tasks (e.g. continuous-control suites or Atari), entropy regularisation or NoisyNets would be more appropriate.

Optional extension

We did not implement the optional chess-playing extension. Our submission focuses on producing a high-quality, well-documented LunarLander solution as required by Problems 1 and 2.

References

- Mnih et al. (2015). Human-level control through deep reinforcement learning. *Nature*, 518.
- van Hasselt, Guez, Silver (2016). Deep Reinforcement Learning with Double Q-learning. *AAAI*.
- Wang et al. (2016). Dueling Network Architectures for Deep Reinforcement Learning. *ICML*.
- Hessel et al. (2018). Rainbow: Combining Improvements in Deep Reinforcement Learning. *AAAI*.
- Fortunato et al. (2018). Noisy Networks for Exploration. *ICLR*.
- Pathak et al. (2017). Curiosity-driven Exploration by Self-supervised Prediction. *ICML*.
- Burda et al. (2019). Exploration by Random Network Distillation. *ICLR*.
- Sutton, Barto (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press.
- Farama Foundation. *Gymnasium documentation: LunarLander environment*.